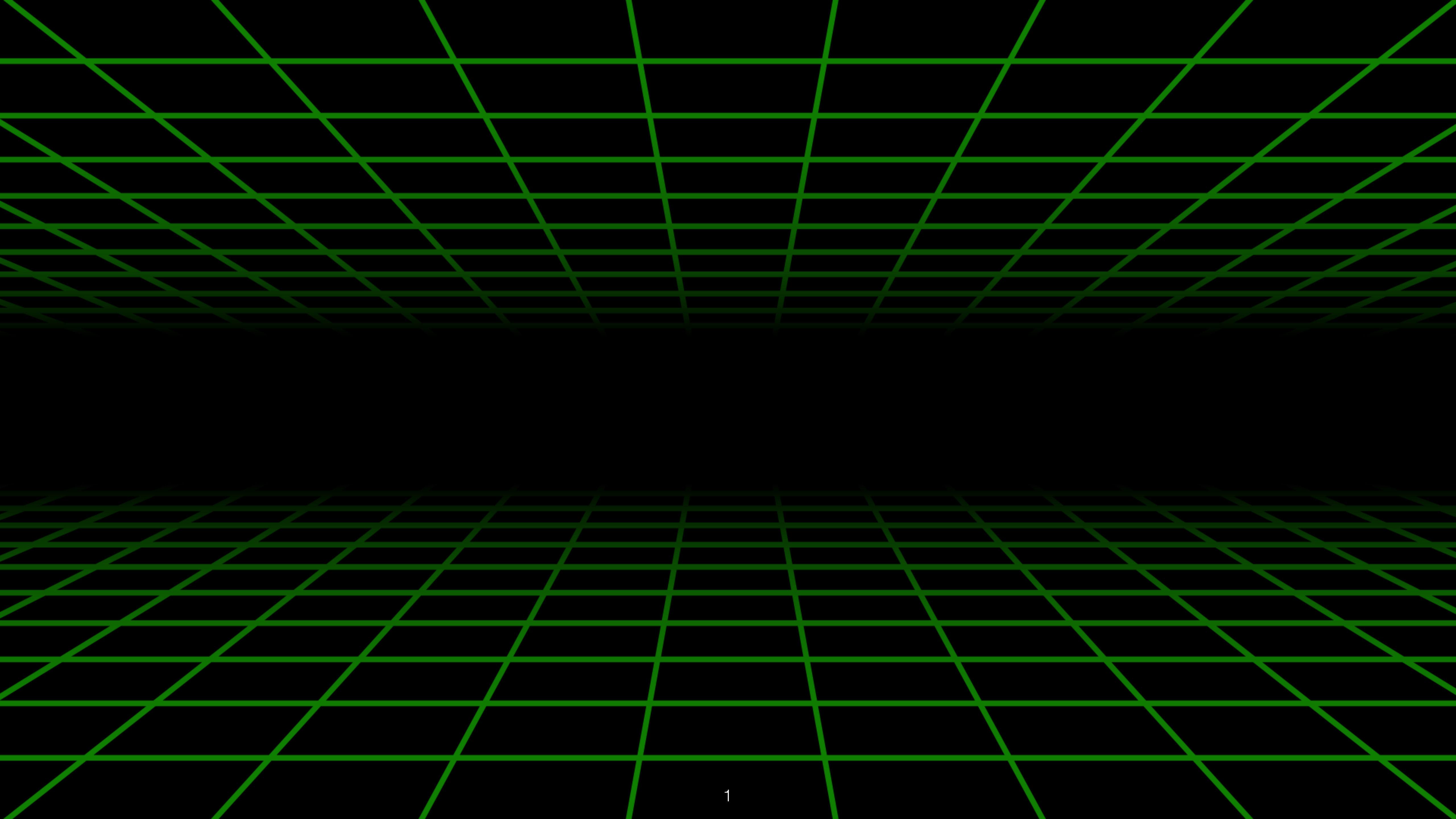




Our Method To Deal With The Madness Of Performance

Hi!
We Are:

Eduardo



Jamie Pond

Hi!
We Are:



Scott Bruce

Hi!
We Are:



Objective

Methods Toward Good Performance

Outline

- Check some benchmarks of code we made versus serious efforts made by GLIBC:
 - Surprises and lessons
- Looking deeper into the code techniques that gave better performance
- Looking deeper into the software engineering techniques that produced work-relieving and performance-enhancing code

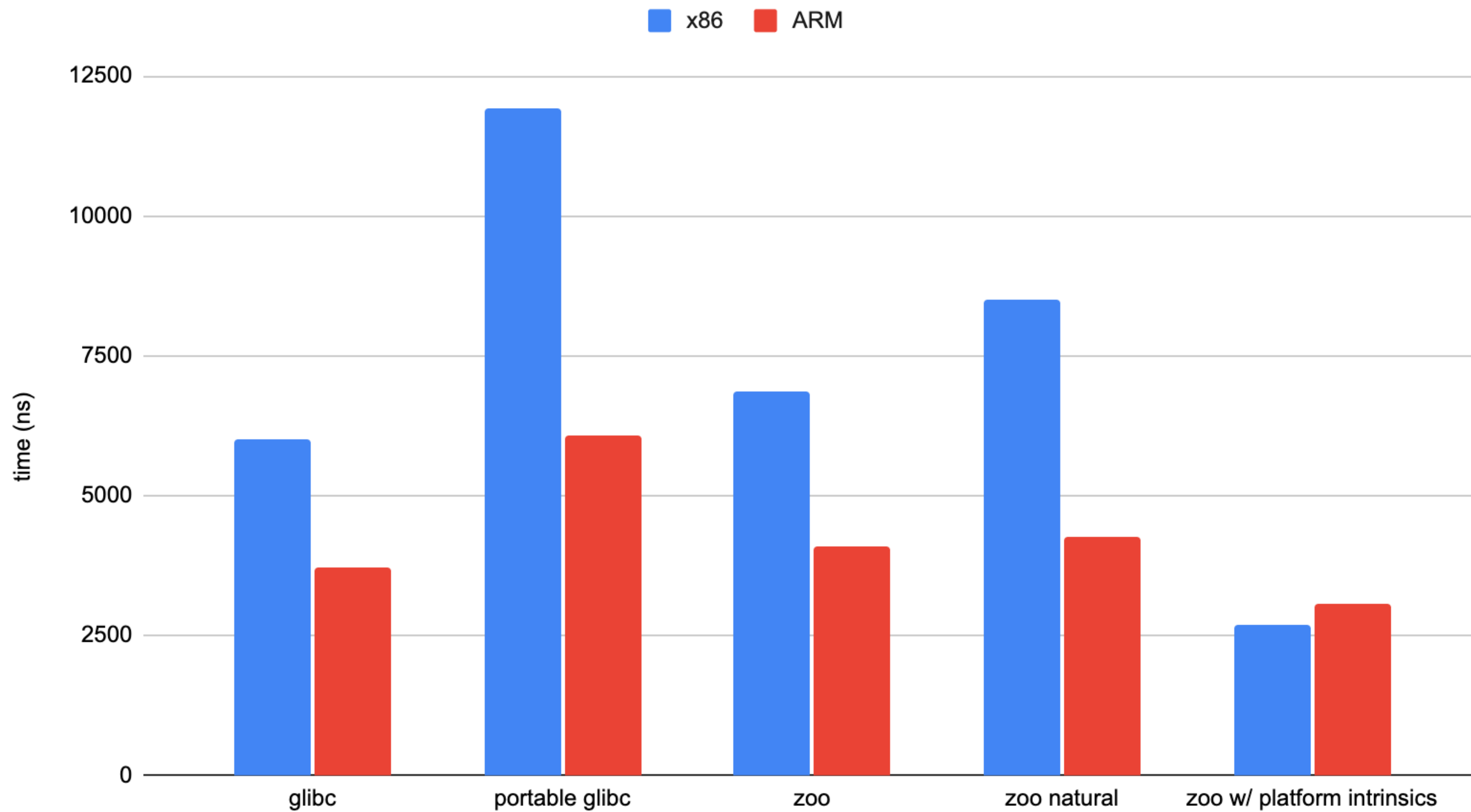
Why we did this work

- A demo to show people how to use our SWAR library
- I wanted to make sure our methods keep the performance of handcrafted SWAR by comparison against something **real**
- String operations are ideal:
 - Implemented in a classic, serial way for 3 decades
 - Not hostile to parallelism either
- Well known
- Well implemented: GLIBC that literally runs the world

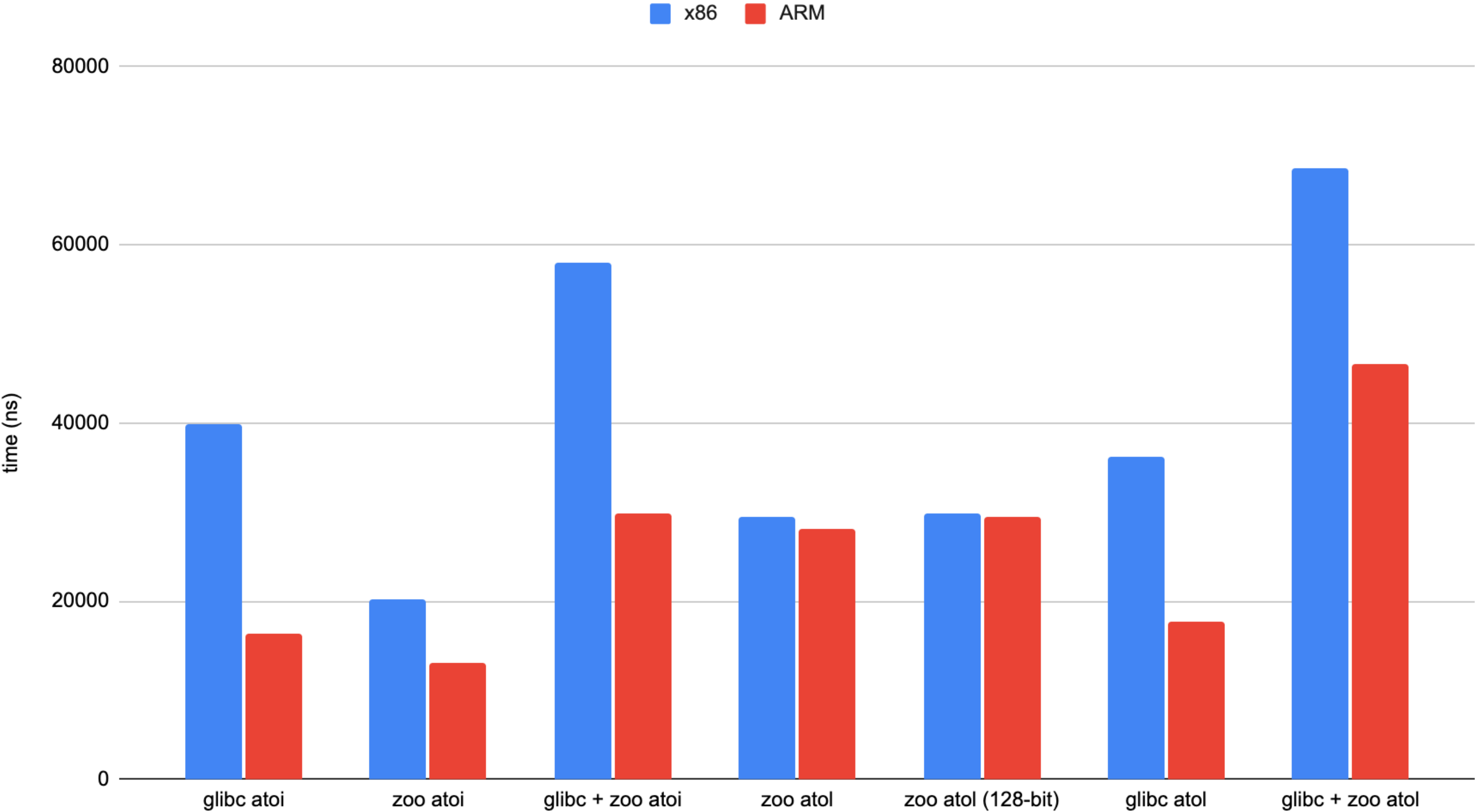
Benchmark	Time		CPU		Iterations
runBenchmark<CorpusLeadingSpaces, InvokeGLIB_Spaces>	35790	ns	35777	ns	18674
runBenchmark<CorpusLeadingSpaces, InvokeZooSpaces>	20545	ns	20537	ns	35738
runBenchmark<Corpus8DecimalDigits, InvokeLemire>	1874	ns	1874	ns	373622
runBenchmark<Corpus8DecimalDigits, InvokeZoo>	1720	ns	1720	ns	406367
runBenchmark<Corpus8DecimalDigits, InvokeLIBC>	12665	ns	12661	ns	54563
runBenchmark<CorpusStringLength, InvokeLIBC_STRLEN>	5569	ns	5567	ns	124866
runBenchmark<CorpusStringLength, InvokeZOO_STRLEN>	6579	ns	6577	ns	99661
runBenchmark<CorpusStringLength, InvokeZOO_NATURAL_STRLEN>	6808	ns	6803	ns	100047
runBenchmark<CorpusStringLength, InvokeGENERIC_GLIBC_STRLEN>	10537	ns	10533	ns	67749
runBenchmark<CorpusStringLength, InvokeZOO_AVX>	2376	ns	2376	ns	287489
runBenchmark<CorpusAtoi, InvokeGLIBC_atoi>	35106	ns	35099	ns	19625
runBenchmark<CorpusAtoi, InvokeZOO_ATOI>	10637	ns	10634	ns	64833
runBenchmark<CorpusAtoi, InvokeCOMPARE_ATOI>	45735	ns	45723	ns	15510
runBenchmark<CorpusStringLength, RepeatZooStrlen>	6847	ns	6846	ns	98706

InvokeLIBC_STRLEN>	5569	ns
InvokeZ00_STRLEN>	6579	ns
InvokeZ00_NATURAL_STRLEN>	6808	ns
InvokeGENERIC_GLIBC_STRLEN>	10537	ns
InvokeZ00_AVX>	2376	ns

strlen comparison



atoi/atol comparison



`zoo::c_strToI` **is over 3 times faster than** `atoi`

...without any hardware SIMD

strlen Key Results

- A very deliberately made set of benchmarks to avoid pitfalls:
 - Everything random (string lengths, data, patterns, etc)
 - Uses prime number counts so that processors won't learn patterns easily
 - Catch2-style benchmarks replicate g-bench
 - No adversarial bias

strlen Key Results

- We made strlen:
 - Portable “natural”
 - Portable SWAR with one little extra trick that is better
 - Proper SIMD for ARM Neon (16 bytes) and x86-64 AVX2 (32 bytes)

Our Surprise!

strlen Key Results

- Our implementations **beat Glibc always**
- In particular, our portable implementation is close to the same performance of ARM Neon Glibc!

Zoo “natural” strlen


```

std::size_t c_strLength_natural(const char *s) {
    using S = swar::SWAR<8, std::uint64_t>;
    S initialBytes;
    auto [base, misalignment] = blockAlignedLoad(s, &initialBytes.m_v);
    auto bytes = adjustMisalignmentFor_strlen(initialBytes, misalignment);
    for(;;) {
        auto nulls = zoo::swar::equals(bytes, S{0});
        if(nulls) { // there is a null!
            auto firstNullIndex = nulls.lsbIndex();
            return firstNullIndex + base - s;
        }
        base += sizeof(S);
        memcpy(&bytes.m_v, base, 8);
    }
}

```

```

std::size_t c_strlen_natural(const char *s) {
    using S = swar::SWAR<8, std::uint64_t>;
    S initialBytes;
    auto [base, misalignment] = blockAlignedLoad(s, &initialBytes.m_v);
    auto bytes = adjustMisalignmentFor_strlen(initialBytes, misalignment);
    for(;;) {
        auto nulls = zoo::swar::equals(bytes, S{0});
        if(nulls) { // there is a null!
            auto firstNullIndex = nulls.lsbIndex();
            return firstNullIndex + base - s;
        }
        base += sizeof(S);
        memcpy(&bytes.m_v, base, 8);
    }
}

```

```

std::size_t c_strlen_natural(const char *s) {
    using S = swar::SWAR<8, std::uint64_t>;
    S initialBytes;

    auto [base, misalignment] =
        blockAlignedLoad(s, &initialBytes.m_v);
    auto bytes = adjustMisalignmentFor_strlen(initialBytes, misalignment);
    for(;;) {
        auto nulls = zoo::swar::equals(bytes, S{0});
        if(nulls) { // there is a null!
            auto firstNullIndex = nulls.lsbIndex();
            return firstNullIndex + base - s;
        }
        base += sizeof(S);
        memcpy(&bytes.m_v, base, 8);
    }
}

```



```

std::size_t c_strlen_natural(const char *s) {
    using S = swar::SWAR<8, std::uint64_t>;
    S initialBytes;
    auto [base, misalignment] = blockAlignedLoad(s, &initialBytes.m_v);

    auto bytes =
        adjustMisalignmentFor_strlen(
            initialBytes, misalignment);

    for(;;) {
        auto nulls = zoo::swar::equals(bytes, S{0});
        if(nulls) { // there is a null!
            auto firstNullIndex = nulls.lsbIndex();
            return firstNullIndex + base - s;
        }
        base += sizeof(S);
        memcpy(&bytes.m_v, base, 8);
    }
}

```

```
for(;;) {  
    auto nulls = zoo::swar::equals(bytes, s{0});  
    if(nulls) { // there is a null!  
        auto firstNullIndex = nulls.lsbIndex();  
        return firstNullIndex + base - s;  
    }  
    base += sizeof(S);  
    memcpy(&bytes.m_v, base, 8);  
}
```


Sources: `strlen`

- Either SWAR or SIMD implementations:
 - Load a block, keeping alignment considerations
 - Parallel processing to detect a null
 - When a block has a null, determine where it is.

Sources Recap

- All work with a block of bytes in parallel: Harder to write, but not much:
 - You need to work with potentially misaligned input pointers:
 - To guarantee you don't consume invalid memory, you read aligned
 - Fill bytes not from the input with stuff that does not interfere
- Only 1 conditional branch: invariant test



Natural?

**A tiny improvement,
first observed in GLIBC**

**A tiny improvement,
first observed in GLIBC**

We don't care about all nulls, only the first

“Pseudo-Boolean SWARs”, Scott Bruce, 2024

$\{ A \mid B \mid 0000 \mid C \} -$

$\{ 1 \mid 1 \mid \quad 1 \mid 1 \}$

$\{ ?? \mid ?? \mid 0-1-X_C \mid C-1 \}$

The key is the MSB flipping

Our Sources

- Our implementations are **very similar**.
- For C++ On Sea we planned to have all flavors consolidated into **one template**, but...
 - Not yet, for simplicity. The same for type traversing associative iteration.
- (Scott Bruce) Our SWAR library is organically transitioning into encapsulating:
 - Software SIMD: more freedom
 - Hardware SIMD: higher performance
 - Mix
 - GPU
 - We are rendering the name “swar” outdated
- Forewarning: **We keep improving** something already best of its kind, discovering deeper insights.

GLIBC Portable SWAR strlen

<https://sourceware.org/git/?p=glibc.git;a=blob;f=string/strlen.c;hb=6d7e8eda9b85b08f207a6dc6f187e94e48172>

```

/* Computing (longword - lomagic) sets the high bit of any corresponding
   byte that is either zero or greater than 0x80. The latter case can be
   filtered out by computing (~longword & himagic). The final result
   will always be non-zero if one of the bytes of longword is zero. */
himagic = 0x80808080L;
lomagic = 0x01010101L;
if (sizeof (longword) > 4)
{
    /* 64-bit version of the magic. */
    /* Do the shift in two steps to avoid a warning if long has 32 bits. */
    himagic = ((himagic << 16) << 16) | himagic;
    lomagic = ((lomagic << 16) << 16) | lomagic;
}
if (sizeof (longword) > 8)
    abort ();

/* Instead of the traditional loop which tests each character,
   we will test a longword at a time. The tricky part is testing
   if *any* of the four* bytes in the longword in question are zero. */
for (;;)
{
    longword = *longword_ptr++;

    if (((longword - lomagic) & ~longword & himagic) != 0)

```


GLIBC Sources - Portable

- They refer to “magic”: The code looks and feels **ad-hoc**.
- The advanced and critical techniques deserve good explanations
- This type of code is rigid and intimidating
- Cascading if-then-else to determine the exact position: **monopolizing the branch predictor resources for minimum benefit:**
- UNDERAPPRECIATED PERFORMANCE PITFALL

GLIBC Sources - SIMD

- AVX: https://sourceware.org/git/?p=glibc.git;a=blob;f=sysdeps/x86_64/multiarch/strlen-avx2.S;h=36ecdfe83cd973fe2219eb30de1142cccb336aa1;hb=HEAD
- NEON: <https://sourceware.org/git/?p=glibc.git;a=blob;f=sysdeps/aarch64/strlen.S;h=ab2a576cdb5665e596b791299af3f4abecb73c0e;hb=HEAD>
- Nothing like each other:
 - Handcrafted assembly
 - The use of `VPMINUB`: “Save some code size by microfusing `VPMINU` with the load”
 - Saving size while they unroll manually 4-wise what already is 32-byte-wise?
 - Suspicious inefficiencies:

```
vmovdqa 1(%rdi), %ymm1
```

```
VPMINU (VEC_SIZE + 1)(%rdi), %ymm1, %ymm2
```

```
vmovdqa (VEC_SIZE * 2 + 1)(%rdi), %ymm3
```

```
VPMINU (VEC_SIZE * 3 + 1)(%rdi), %ymm3, %ymm4
```

- Why keeping `rdi` misaligned?

Their Sources - SIMD

- ARM - NEON Critical line: `shrn maskv.8b, maskv.8h, 4`
- I did not know how you would craft a boolean SIMD in ARM
- I explained profusely: <https://github.com/thecppzoo/zoo/blob/f28eb4337976a98e4a7b6d28874f4925be4ff17a/benchmark/atoi.cpp#L493>

Their Sources - SIMD

- Evidence of a whole lot of effort
 - Loop unrolling (4-wise in AVX2 32-bytes: 128 bytes), 2-wise in ARM 16 bytes: 32 bytes: **contradictory**.
- Ad-hoc
- How would you improve this code?

Our sources in the Compiler Explorer

- <https://godbolt.org/z/rs3bMPeYs>
- Elegant?

GLIBC `atoi`: Not SIMD

- We choose to not explain it
- Conversion of alpha to integer is more involved:
 - Leading spaces
 - Optional sign
 - Insignificant zeros
 - Non-digit character terminator

Core of atoi

- Converting groups of bytes
- Change of endianness: input comes in “Big Endian”, our platforms are in “Little Endian”
- Detecting the end of input numbers along with the conversion
- We provide two versions: 64 bit and 128 bit

Our `atoi` Prof. Lemire Parsing in Compiler Explorer

- Converting 8 bytes of input in pseudo-Big Endian to a little endian integer: Lemire
- Our Lemire
- Our Lemire for 16 bytes!
- Elegant?
- Easier to use?
 - See how it composes

What Performs Well?

Caches

- Space and time localities
- Cache line
- Cache associativity mapping: **False Sharing**.
- The importance of space-efficiency in data as well as object code
 - Loop unrolling is strongly disadvantageous:
 - **CLANG: I AM LOOKING AT YOU!**
- The impact of inefficiency is impossible to even detect in naive benchmarks:
 - **Thou shalt not do naive benchmarks.**

Out of Order/Speculative Execution, Branch Prediction

- Pipeline architecture
- Processors today can execute 7, 8 instructions simultaneously
- The Physical Register File has more than 300 actual registers for the ~16 “architectural registers”
- Lots of **speculative** execution
 - Committing the results to the program state: “retiring instructions”
- The key is good branch prediction!

Keeping the Branch Predictors Happy

- Make your code unconditional as much as possible (avoid if statements where possible!)
- Convert control dependencies into data dependencies
- This means clever mathematics
 - Example: Scott's Saturated Addition, https://github.com/thecppzoo/zoo/blob/f28eb4337976a98e4a7b6d28874f4925be4ff17a/inc/zoo/swar/associative_iteration.h#L349C3-L356C2 which is simply two's complement addition
 - Jamie's SWAR exponentiation, zoo's SWAR multiplication.
- Speculative, out of order execution resources get you covered

Keeping the Branch Predictors Happy

- Cascading conditional branching: extremely hostile to performance
 - Coalesce several conditions into one table jump (one `switch`, or jumping to a function pointer):
 - Jody Hagins and my compile-time-to-runtime (what `std::variant` does).
- Branch predictor “slots” must suffer from the same mapping problem as cache lines: **FALSE SHARING!**
- **USE EXCEPTIONS!**
 - Error codes and the “Result” or “Expected” monad bake-in gazillion conditional branches that in practice is never taken: dramatically maximize the chances for false sharing!
- **Branch predictor behavior is notoriously hard to measure:** How do you know your processor is not learning your benchmark pattern?

Quantum Physics & Microbenchmarking!

Quantum Physics and Microbenchmarking

- The measurement disturbs the experiment!
- Personally, I rely more on careful analysis than benchmarking:
 - Emery Berger proves how incredibly noisy benchmarking is:
 - <https://www.youtube.com/watch?v=r-TLSBdHe1A>
 - -O2 and -O3 are statistically insignificantly different
 - Performance suffers from artifacts such as object, memory layouts: Emery Berger, Krister Walfridsson
 - <https://kristerw.blogspot.com/2018/10/optimization-and-performance-measurement.html>

Quantum Physics and Microbenchmarking

The measurement disturbs the experiment!

Microbenchmarking

- The measurement disturbs the experiment!
- Personally, I rely more on careful analysis than benchmarking:
 - Emery Berger proves how incredibly noisy benchmarking is:
 - <https://www.youtube.com/watch?v=r-TLSBdHe1A>
 - -O2 and -O3 are statistically insignificantly different
- Performance suffers from artifacts such as object, memory layouts: Emery Berger, Krister Walfridsson
 - <https://kristerw.blogspot.com/2018/10/optimization-and-performance-measurement.html>

Microbenchmarking

- The evidence in benchmarks is to test your model of performance!
- Blind, **or automatic!** application of benchmarks is counterproductive
- Your model must take into account that what works well today could zap performance later:
 - Loop unrolling
- The “what performs well” knowledge in Zoo tries to be ahead of its time, it predicts what will run well.
 - Example: butterfly networks.
 - My algorithm to traverse sets

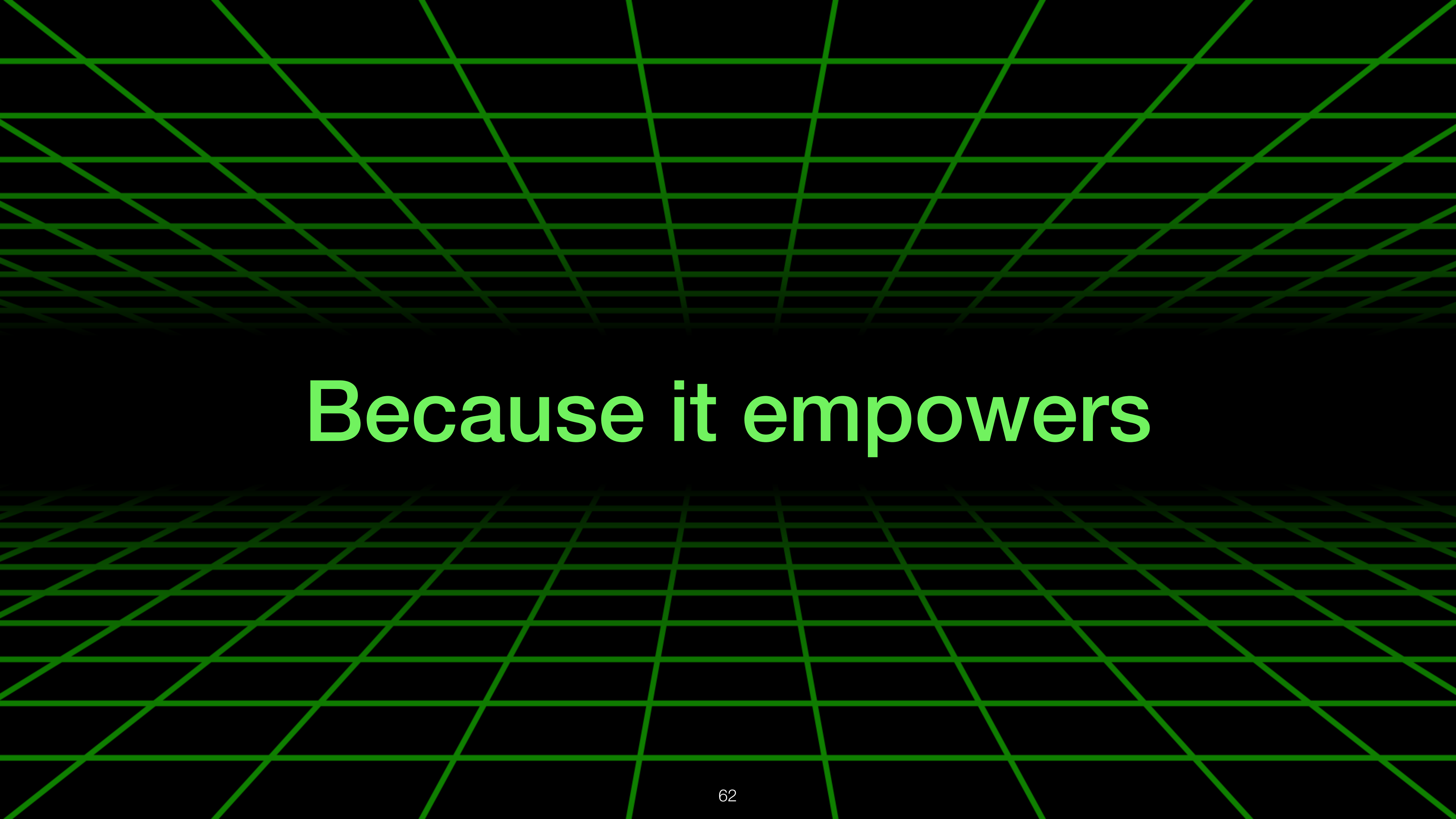
“Bottling” Performance Knowledge



Via extreme success at composability

Epistemological Doctrine of Software Engineering

Knowledge Composes Exponentially



Because it empowers

Engineer Software As Expressing Knowledge in Source Code

Composability: The Single Responsibility Principle

The Single Responsibility Principle:

A goal, not a method

“There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult.”

–C.A.R. Hoare, the inventor of Quicksort, at his Turing Award lecture

**“[S]implicity and elegance are unpopular
because they require hard work and
discipline to achieve and education to be
appreciated. Besides, complexity sells
better.”**

–Edsger Dijkstra, at his own Turing Award lecture

**Complexity sells better because it
proves work**

**The value of simplicity is
counterfactual**

Method to Achieve The Single Responsibility Principle

- Study the way you learn a topic:
 - There will be **concepts**.
- Deconstruct the concepts as hard as you can.
- Express those concepts in software
- Your expression of concepts will initially be bad, but **ITERATE**: the use of your concepts are now their own knowledge
- There are tools to help:
 - Unit testing are by themselves an excellent exercise in decomposition, deconstruction
 - It is not the puerile benefit of knowing your components do what they are supposed to do when used in isolation, but a powerful incentive to **deconstruct complexity into simplicity**.

Method

- If you want quick results, do less of a depth first search and more of a breadth first search:
- These are sigmoids, how you minimize the time to achieve a goal multiplying sigmoids

Method

- Only the Generic Programming Paradigm preserves performance:
 - Object Orientation is counterproductive and un-sound
 - Functional Programming allows the expression of arbitrarily higher order knowledge directly, but by miserable performance
- With effort one translates from higher order knowledge into concrete C++
 - Maximum level of decomposition of concepts -> Single Responsibility Principle -> maximum software component composability -> maximum user benefit

Anti-Method

- Copy & pasting:
 - Easy to understand
- Contrary to every principle in the Epistemological doctrine of software engineering:
 - Hostile to deeper understanding of concepts and especially the relations between similar concepts, or especially hostile to organic generalization

“[p]remature optimization is the root of all evil”

–Nasty misquoting of Donald Knuth

“We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%. ”

–What he actually said, 1974 “go to” paper

Premature

Pre-mature: before you know

Premature Optimization: Premature Commitment

“Premature commitment is a nasty evil”

–My advocacy

Anti-Method

- The delusion that software progress is linear, not exponential, leads to premature commitment to poorly understood concepts.
- Your software engineering method does not make the central point of the process the production of knowledge to be expressed in the source code as soon as possible:
 - In particular, the process allows early irreversible decisions
 - The good method is to under-specify! To postpone decisions for as long as possible

Method

- No explicit typing!: truly **almost always** `auto`!
 - Allows the code to be consistent, especially great for experimentation
 - Allows the code to absorb new knowledge
 - You use strong typing for pattern matching, not as a crutch for understanding
 - Promotes a rough understanding of the characteristics of everything:
 - Sufficient understanding to do things
 - Insufficient to over-specify
 - The tension leads to superior understanding of what the values/code ought to be

Zoo doesn't have class

- I find humorous that people use `class`. As if they were able to sharply distinguish between the private and public interfaces of types.
- The understanding about types ought to improve organically.
- The understanding of what is very public versus very private of types can only emerge after a lot of use, understanding.
- It is natural to have APIs that are in-between.
- As users of types done by others, it is our responsibility to understand:
 - When we are messing with things we don't understand
 - When we are limiting out choices (premature commitments)



Basically, a very strange method

Living in an exponentially improving codebase

Exponential Growth Feels Different

- Software writers often mistake writing lots of code for doing the correct thing.
- Writing code is not even obviously directionally correct: there is a burden of proof on each additional line that it be STRONGLY net positive.
- The counterargument to 'worse is better': many or most decisions are one way doors.
- Thinking harder, doing more exploration, and writing less code is harder than and 'feels' less productive than writing lots of code fast.
- The result is that each 'unit improvement' of the codebase feels harder, but provides exponential improvements.
- The comparative advantage of this method is enormous: almost any action should be taken to preserve exponential growth.

Living in an exponentially improving codebase

- We practice a high frequency “Electromagnetic Wave Of Progress”: we go routinely from practice to theory to practice.
- We don’t overspecify, we struggle to identify the things we are certain, and build upon them
- We don’t waste effort into uncertain things, you would not see me using explicit typing if I can avoid it, you would not see my code unrolling loops manually, you would not see me applying automatically the results of benchmarks.
- You will see the ugliest source code you’ve ever seen, because we fight the limitations of the expressive capabilities of C++ to encode subtle knowledge, and the subtle knowledge abounds.
- You will feel despair about the depth of the you-need-to-understand concepts in this codebase

**Our method is to meet the madness of
performance with our own madness**



BUT IT IS SO FUN!

equals

[0101 | 0000 | 1000 | 0011] = a^b: different lanes will be non-zero

[1000 | 1000 | 1000 | 1000] = SWAR<4, uint16_t>::MostSignificantBit

[**1**101 | **1**000 | **1**000 | **1**011] = (a^b) | TAOC_P_H, subtract lanes of 1:

[0001 | 0001 | 0001 | 0001] = SWAR<4, uint16_t>::LeastSignificantBit

[1??? | **0**??? | **0**??? | 1???]

Discard false positives by disjunction with a^b: different

Flip different: equals.

Mask with MSB to make it a booleanSWAR